Chapter 7
Performance Libraries

Abstract In this chapter we discuss performance optimization libraries which concentrate on memory issues. We first describe the Google Perftool set of tools which include the `tcmalloc` memory allocation and thread-key based memory allocation library. Perftools also contains a heap-checker and heap-profiler, their usage alongwith examples which are optimized by their usage is shown in this chapter. We compare the performance of perftools memory allocation with APR and the results are presented in this chapter. Another technique for memory optimization is the use of garbage collection. This is included in Java and Common Lisp family of languages, but using the Boehm garbage collector, we also use garbage collection (GC) in C and C++ programs. Examples using Boehm GC are presented in this chapter, and we investigate the impact of GC on performance and memory consumption.

Contents

7.1 Google perftools 145
7.2 Boehm GC : garbage collection 151
7.3 Using Boehm GC 151
7.4 Conclusion 154

7.1 Google perftools

Google's *perftools* libraries are an open-source performance enabling API for memory allocation, leak checking, and profiling. Version 1.5 of Google perftools include:

1. `tcmalloc`: thread caching memory allocator,
2. `heap_checker`: heap checking and leak detection API,
3. `heap_profiler`: performance monitoring of heap allocation,
4. `cpu_profiler`: performance monitoring for functions.

We discuss these below.

S. Koranne, *Handbook of Open Source Tools*,
DOI 10.1007/978-1-4419-7719-9_7, © Springer Science+Business Media, LLC 2011

7.1.1 perftools : tcmalloc

Google Perftool tcmalloc is designed as a fast, optimized thread aware replacement for C lib standard malloc. On multi-threaded systems, tcmalloc reduces thread contention by reducing threading overhead for small objects, and by using fine grained thread aware locking. While standard malloc also uses thread specific arenas for memory allocation, migration of free space from one thread arena to another is not implemented in the standard C library, while perftools tcmalloc reduces this wastage of memory space. Another advantage of tcmalloc is the optimized representation of small objects. To make use of tcmalloc in their own code, the application can either link to -ltcmalloc or use LD_PRELOAD to load the tcmalloc shared library (this is the way to use tcmalloc in applications which cannot be recompiled to take advantage of tcmalloc, which should not be a problem for the software considered in this book).

7.1.1.1 Implementation

Perftool tcmalloc assigns each thread a local cache from which small allocations are directly serviced. A page is divided into a sequence of small objects, while a large object is allocated directly from the main heap. tcmalloc uses a class of 60 allocatable size-classes. Object size requests outside this range are rounded up to the nearest available size. Since thread local arenas are available for small objects, if the *free list* for an arena is not empty, an object of the appropriate size is returned directly (no locking is required). If the free list for the requested size class is empty, a collection of objects from a central free list is requested and placed on the thread local free list, subsequently an object from this list is returned to the application. If the central list is also empty, then a run of pages is carved up into objects of the requested class and placed on the lists. Since the API is identical to malloc for use, it can be quickly integrated with existing code.

On an existing discrete geometry application using STL in C++, standard GNU C malloc had a runtime of : 266.12 seconds. Linking the same application with tcmalloc reduced the runtime to 208 seconds. Another example is shown in Figure 7.1 where the runtime performance of GNU C malloc is compared to perftools tcmalloc.

7.1.2 perftools : heap_checker

The heap checker heap_checker is part of tcmalloc so its use is already possible by linking the libtcmalloc library with the application. However, linking to libtcmalloc does not turn on heap checking. The heap checker tracks usage of memory before main() and confirms that all allocated memory has been freed at exit(). If the heap checker detects any memory that has been allocated but not

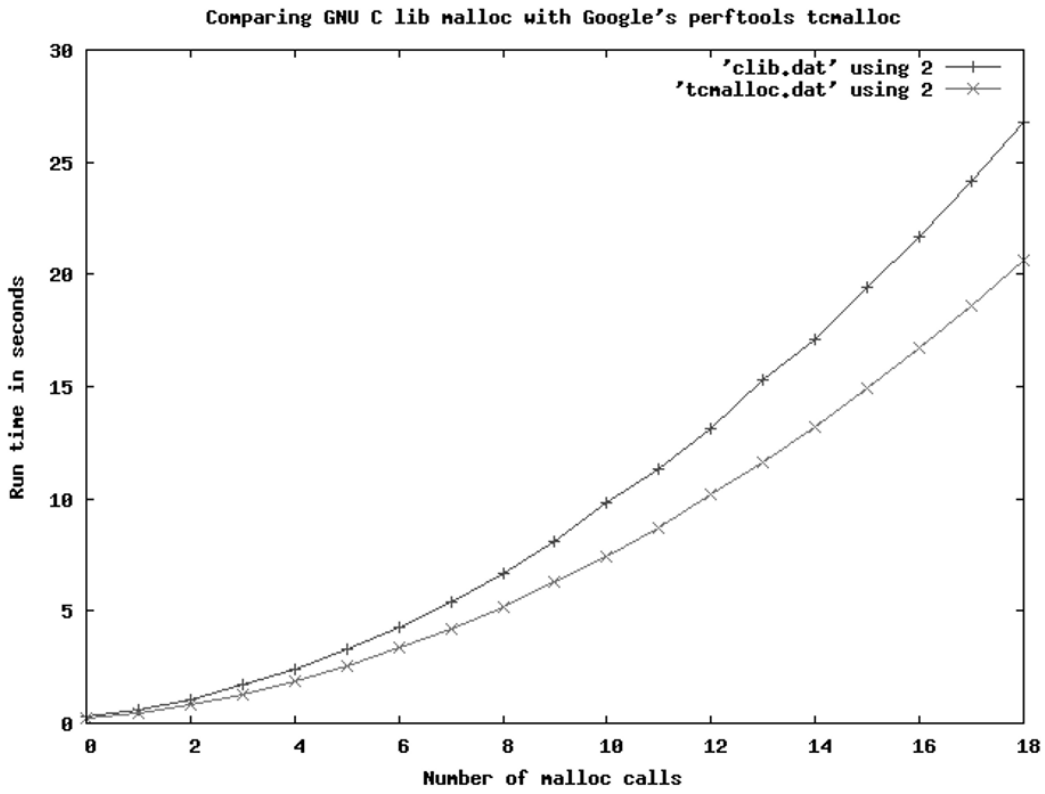


Fig. 7.1 Performance measurement of GNU C `malloc` with perftools `tcmalloc`.

freed at `exit()` it aborts the program and prints a message. It should be noted that `heap_checker` records the stack trace for each allocation (thus increasing the memory usage and runtime of the application significantly).

To switch on the `heap_checker`, the environment variable `HEAPCHECK` should be set one of the following: (i) `minimal`, (ii) normal, (iii) strict, and (iv) draconian. Example of `minimal` output is shown below:

```
sizeof(Object) = 72

Leak check _main_ detected leaks of 3120 bytes in 101 objects
The 2 largest leaks:
Leak of 2400 bytes in 100 objects allocated from:
@ 8048979
@ 94ebb6
@ 8048801
Leak of 720 bytes in 1 objects allocated from:
@ 80488e8
@ 94ebb6
@ 8048801
```

It should be noted that since the heap-checker uses the heap-profiling framework (see Section 7.1.3) internally, it is not possible to use both tools at the same time. To disable the heap-checker for a block of code we can use the following method:

```
// code block has know memory allocation
{
  HeapLeakChecker::Disabler myDisabler;
  int *n = new int[1024];
  if( wanted_elsewhere) globalArray = n; //
  // Or we can use
  IgnoreObject( n ); // which also waives the heap error
}
```

In addition to basic heap-checking, it is possible to check for memory alignment problems (on modern processors, memory latency can be heavily influenced by memory alignment). The environment variable `HEAP_CHECK_TEST_POINTER_ALIGNMENT` can be set to enable this check.

7.1.3 perftools : heap_profiler

The primary uses of `heap_profiler` are to detect places that perform a lot of memory allocation in C++ program code. It can also be used to detect memory leaks. Similar to the HEAP checker, the heap profiler does not switch on by default. To switch on the heap profiler, the environment variable `HEAPPROFILE` needs to set to a file location. It is possible to analyze portions of code for heap-profiling as:

```
// performance sensitive code block
void RegenrateMatrix( double* data, unsigned int N ) {
  // do computation
  #ifdef _ENABLE_PROFILING_
  if( IsHeapProfilerRunning() ) HeapProfilerDump();
  #endif
}
void CheckStrata(double* data, unsigned int N) {
  #ifdef _ENABLE_PROFILING_
  HeapProfilerStart(``/tmp/strata_profile'');
  #endif
  // algorithm phase I, setup FFT vectors
  #ifdef _ENABLE_PROFILING_
  HeapProfilerDump();
  #endif
  // call actual complex function
  RegenrateMatrix( data, N );
  #ifdef _ENABLE_PROFILING_
  HeapProfilerStop();
  #endif
}
```

The function `HeapProfilerStart` expects the filename prefix as the argument. The function `IsHeapProfilerRunning` can be used to check if the function is already being profiled. Once the profile data has been generated into a file the `pprof` tool can be used to analyze the data and generate reports. The reports can be generated using TEXT formar, or PostScript output (as shown in Figure 7.2.

The TEXT format report example is shown below:

```
$ ~/OSS/bin/pprof --text poly_enum_i686.tcmalloc
   /tmp/enum_profile.0001.heap
Total: 6.8 MB
```

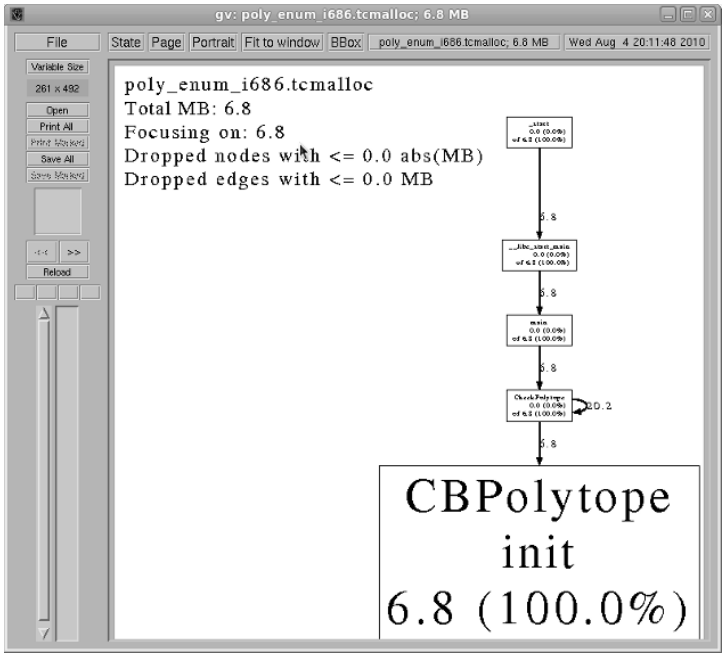


Fig. 7.2 Google Perftools `pprof` output graph

```
 6.8 100.0% 100.0%      6.8 100.0% CBPolytope::init
 0.0   0.0% 100.0%      0.0   0.0% nauty
 0.0   0.0% 100.0%      0.0   0.0% refine1
 0.0   0.0% 100.0%      0.0   0.0% firstpathnode0
 0.0   0.0% 100.0%      0.0   0.0% doref
 0.0   0.0% 100.0%      0.0   0.0% bestcell
 0.0   0.0% 100.0%      0.0   0.0% CBPolytope::compute_orbits
 0.0   0.0% 100.0%      6.8 100.0% CheckPolytope
```

At least in this example, the polytope `init()` function does all the memory allocation (which is expected). We can also ask `pprof` to focus on specific functions using the `--focus=<name>` command-line option:

```
$ ~/OSS/bin/pprof --focus=nauty --text
       poly_enum_i686.tcmalloc /tmp/enum_profile.0001.heap
Total: 6.8 MB
     0.0   0.0%   0.0%      0.0   0.0% nauty
     0.0   0.0%   0.0%      0.0   0.0% refine1
     0.0   0.0%   0.0%      0.0   0.0% firstpathnode0
     0.0   0.0%   0.0%      0.0   0.0% doref
     0.0   0.0%   0.0%      0.0   0.0% bestcell
```

7.1.4 perftools : `cpu_profiler`

To add CPU profiling to the application, we have to link with the `lprofiler` library of perftools. Similar to the heap profiling, this does not switch on the profiling code, it only links the code. To enable CPU profiling, the environment variable `CPUPROFILE` needs to set to the filename where the cpu profile results will be generated. The environment variable `CPUPROFILE_FREQUENCY` and `CPUPROFILE_REALTIME` control the behavior of this profiling. Once the data has been generated the `pprof` tool can be used to analyze the trace, as shown below:

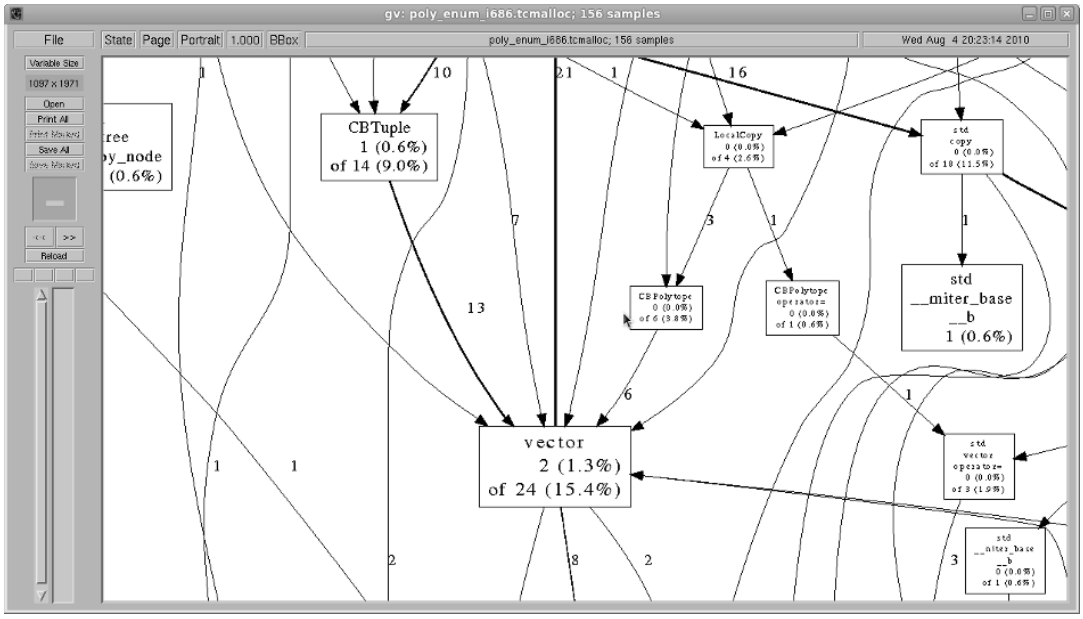


Fig. 7.3 Google Perftools with CPU profiler output graph

```
$ ~/OSS/bin/pprof  --text poly_enum_i686.tcmalloc
       /tmp/enum_profile
Removing _init from all stack traces.
Total: 156 samples
  48  30.8%  30.8%  48  30.8% __mcount_internal
  25  16.0%  46.8%  71  45.5% mcount
  12   7.7%  54.5%  12   7.7% _init
   8   5.1%  59.6%  49  31.4% CBPolytope::init
   5   3.2%  62.8%   5   3.2% __i686.get_pc_thunk.bx
   5   3.2%  66.0%   5   3.2% std::vector::operator[]
   3   1.9%  67.9%   3   1.9% std::_Rb_tree_increment
   3   1.9%  69.9%   6   3.8% std::__copy_move::__copy_m
   3   1.9%  71.8%  17  10.9% std::__copy_move_a2
   2   1.3%  73.1%   2   1.3% refine1
   2   1.3%  74.4%   4   2.6% std::_Rb_tree::_M_destroy_node
   2   1.3%  75.6%  11   7.1% std::_Rb_tree::_M_insert_unique
   2   1.3%  76.9%   2   1.3% std::_Rb_tree::_S_value
```

```
   2   1.3%  78.2%    8    5.1% std::set_difference
```

As with the heap-profiler, it is possible to restrict the report to match a regular expression string using the `--focus` command-line option. We can generate the graphical report for the CPU profiler as well, as shown in Figure 7.3.

7.2 Boehm GC : garbage collection

Boehm Garbage Collector (GC) is a conservative garbage collection for C and C++. Garbage collection can be defined as the automatic reclaiming of dynamic memory which is no longer in use (referenced) by any object in the program. Use of garbage collection was pioneered in the Common Lisp community, which also contributed large advances in the field of garbage collection, mostly in performance and capabilities.

GNU Compiler Collection uses the Boehm GC internally. The Java language also uses garbage collection. The use of garbage collection not only frees the developer from the responsibility of keeping track of dynamic variable lifetime, as well as prevent leaks and improves system performance (when used properly). When the developer understands that dynamic memory can be reclaimed automatically, better algorithms can be chosen (which without GC would be too complicated to implement correctly). This can counteract the slight performance and capacity hit that GC entails. Since the system has to recognize all dynamic memory as alive or not, extra information (meta-data) has to be kept for dynamic memory. Boehm GC is implemented to reduce this impact. Moreover, the Boehm GC can also be used as a leak detector.

7.3 Using Boehm GC

Boehm GC's API have been designed as a replacement for the C library `malloc`. The basic idea behind garbage collection is that all valid dynamic memory must be pointed to by some object in the program. Such an object can be (i) other dynamic memory, (ii) stack variables, (iii) data, (iv) statically allocated BSS segments, or (v) registers. Pointer recognition is a fundamental problem, and Boehm GC is a conservative GC (which means that it errs on the side of caution) when deciding which memory to reclaim. Memory allocated by system `malloc` are not seen by the GC so mixing memory allocations is not a good idea.

The C interface of the allocator is listed below:

- GC_malloc:
- GC_malloc_atomic:
- GC_realloc:
- GC_free:

A simple way to use Boehm GC is to define:

```
#define malloc(n) GC_malloc(n)
#deine calloc(m,n) GC_malloc((m)*(n))
```

There is also a C++ interface to the garbage collector. A garbage collector is most useful when the lifetime of the object cannot be easily tracked. Maintaining reference count within objects makes the code complicated and error prone. Using automatic garbage collection can solve this problem. Consider a small simulation kernel as shown in Listing 7.1.

```
// \file unknown_lifetime.cpp
// \author Sandeep Koranne, (C) 2010
// \description Example of using garbage-collection as object
// life time is not known apriori
#ifdef USE_GC
#include <gc.h>               // Boehm garbage collector
#endif
#include <fstream>            // reading /proc/self/statm
#include <iostream>           // program IO
#include <cassert>            // assertion checking
#include <cstdlib>            // exit
#include <cstdio>             // perror
#include <sys/resource.h>     // setrlim

static unsigned long num_mallocs = 0;
#ifdef USE_GC
  #define MALLOC GC_malloc
#else
  #define MALLOC malloc
#endif

struct Node {
  struct Node *left, *right;
  int data;
  int mark;
  char BUF[10240];
  Node() : left( NULL ), right( NULL ), data( 0 ), mark( 0 ) {}
  void reset() { left = right = NULL; data = mark = 0; }
};

std::ostream& operator<<( std::ostream& os, const Node& N ) {
  return os << N.data << "\t" << N.mark;
}

Node ** ConstructNodes( unsigned int N ) {
  Node ** retval = (Node**)MALLOC( N * sizeof( Node * ) );
  assert( ( retval != NULL ) && "retval allocated" );
  for( int i=0; i < N; ++i ) {
    retval[i] = (Node*)MALLOC( sizeof( Node ) );
    retval[i]->reset();
    retval[i]->data = i;
  }
  return retval;
}

static void PrintNodeData( Node **data, unsigned int N ) {
  for( int i=0; i < N; ++i )
    std::cout << (*data[i]) << std::endl;
}

static void RunSimulation( Node **data,
                           unsigned int N,
                           unsigned int LOOP_COUNT ) {
  for( int i=0; i < LOOP_COUNT; ++i ) {
```

```
    unsigned int source_id = rand() % N;
    unsigned int dest_id   = rand() % N;
    assert( ( data[ source_id ] != NULL ) && "source is invalid" );
    assert( ( data[ dest_id ] != NULL ) && "dest is invalid" );
    Node *source = data[source_id], *dest = data[dest_id];
    Node *temp = (Node*) MALLOC( sizeof( Node ) );
    num_mallocs++;
    temp->reset();
    temp->data = source->data + dest->data;
    if( source->data  < dest->data ) {
      source->mark |= 0x01;
      source->right = temp;
      data[ dest_id ] = temp;
    } else if( source->data > dest->data ) {
      dest->mark |= 0x01;
      dest->left = temp;
      data[ source_id ] = temp;
    } else {
      data[ source_id ] = data[ dest_id ] = temp;
    }
  }
}
static size_t GetMemorySize(void) {
   std::ifstream ifs;
   ifs.open("/proc/self/statm",std::ios::in);
   size_t memoryUsed = 0;
   ifs >> memoryUsed;
   return memoryUsed;
}

static void DisplayMemoryUsed(void) {
   size_t memUsed = GetMemorySize();
   std::cout << std::endl << "Program consumed "<< memUsed << " kb.\n";
}

int main( int argc, char *argv [] ) {
  struct rlimit mem;
  mem.rlim_cur = mem.rlim_max = 102400;
  int rc = setrlimit( RLIMIT_DATA, &mem );
  if( rc ) { std::perror( "setrlimit"); exit( 1 ); }
  unsigned int N = 10;
  Node **data = ConstructNodes( N );
  RunSimulation( data, N, 100000 );
  std::cout << "number mallocs = " << num_mallocs << std::endl;
  DisplayMemoryUsed();
  std::cout << std::endl;
  return 0;
}
```

Listing 7.1 Example of using Boehm garbage collection

We compile this program with and without the garbage collector:

```
g++ -O3  unknown_lifetime.cpp
$ /usr/bin/time ./a.out
number mallocs = 100000
Program consumed 251876 kb.

0.08user 0.94system 0:01.10elapsed 93%CPU
  (0avgtext+0avgdata 0maxresident)k
0inputs+0outputs (0major+102255minor)pagefaults 0swaps
# Now we compile with garbage collection
$ g++ -O3 -DUSE_GC unknown_lifetime.cpp   -lgc
$ /usr/bin/time ./a.out
```

```
number mallocs = 100000

Program consumed 1491 kb.

3.65user 0.01system 0:03.77elapsed 97%CPU
  (0avgtext+0avgdata 0maxresident)k
0inputs+0outputs (0major+625minor)pagefaults 0swaps
```

The garbage collected binary is several times slower (this is a very artificial example where the runtime cost of the memory allocator dominate any processing), but the important aspect is the significant reduction in memory consumption from 251 MB (for the non-gc binary), to 1.5 MB for the garbage collected binary. Although, the example is simplistic (the dangling pointers can be fixed with some extra code and reference counts), having the flexibility of automatic garbage collection allows the programmer to concentrate on the actual simulation kernel in this example, without running out of memory.

7.4 Conclusion

In this chapter we discussed performance optimization libraries which concentrate on memory issues. We first described the Google Perftool set of tools which include the `tcmalloc` memory allocation and thread-key based memory allocation library. We compared the performance of perftools memory allocation with APR and the results are presented in this chapter. Another technique for memory optimization is the use of garbage collection. Examples using Boehm GC are presented in this chapter, and we investigate the impact of GC on performance and memory consumption.

Chapter 8
Compression Engines

Abstract In this chapter we present libraries and APIs for lossless compression. Even with the geometric rise in memory capacity, the information theoretic compression gains provided at relatively cheap computing power can result in impressive performance and capacity gains. In this chapter we present with the help of examples, the use of the ZLIB and BZIP2 libraries. Examples are presented using C/C++ API as well as Python modules. More recently, LZMA and XZ Utilities also provide even better compression, albeit at slightly slower speed. Their use is also described in this chapter.

Contents

8.1 ZLIB Compression Library 156
8.2 LIBBZ2 and BZIP2 162
8.3 LZMA and XZ Utils 163
8.4 Conclusion 164

Although network speeds and memory capacities have expanded at a rapid rate in recent years, application demands for ever more features and capabilities have continued to place a constraint on the total memory space consumed by the program, or data which is communicated between cooperating processes. One solution to alleviating this concern is the use of compression, both lossless (for applications which need bit-equivalent data on both ends of the communication) or lossy (such as image compression). In the next section we discuss techniques for lossless compression including the use of *deflate*, and its implementation in ZLIB. As the performance gap between CPU and memories continue to expand, spending CPU cycles for compression and decompression, which reduces the memory requirement is indeed beneficial.

S. Koranne, *Handbook of Open Source Tools*,
DOI 10.1007/978-1-4419-7719-9_8, © Springer Science+Business Media, LLC 2011